GenAI Unit 3 Lab

Name	Dhanya Prabhu
SRN	PES2UG23CS169
Section	C

Notebook 1: Advanced RAG & Retrieval Enhancement Techniques

Part 1a: Why Naïve RAG Fails — A Live Demonstration

```
Corpus size: 5 documents
Query 1 (semantic): 'How do neural nets learn?'
Query 2 (keyword):  'BM25 term frequency'
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
modules.json: 100% 349/349 [00:00<00:00, 20.5kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 7.02kB/s]
README.md: 10.5k/? [00:00<00:00, 529kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.41kB/s]
config.json: 100% 612/612 [00:00<00:00, 30.8kB/s]
model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 452MB/s]
Loading weights: 100% 103/103 [00:00<00:00, 441.78it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% 350/350 [00:00<00:00, 16.9kB/s]
vocab.txt: 232k/? [00:00<00:00, 8.52MB/s]
tokenizer.json: 466k/? [00:00<00:00, 17.9MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 12.1kB/s]
config.json: 100% 190/190 [00:00<00:00, 19.5kB/s]
=== Dense Retrieval Results ===

Query: 'How do neural nets learn?'
  Rank 1 [doc_4] score=0.7550: Neural networks learn by adjusting weights through backpropagation.
  Rank 2 [doc_3] score=0.3656: Gradient descent is an optimization technique used to minimize the los
  Rank 3 [doc_1] score=0.2918: BERT is a bidirectional encoder trained using masked language modellin

Query: 'BM25 term frequency'
  Rank 1 [doc_2] score=0.5714: The BM25 algorithm ranks documents based on term frequency and inverse
  Rank 2 [doc_0] score=0.1448: Transformers use self-attention mechanisms to process sequences in par
  Rank 3 [doc_4] score=0.1129: Neural networks learn by adjusting weights through backpropagation.
```

Part 1b: Sparse Retrieval — BM25

```
=== BM25 Retrieval Results ===

Query: 'How do neural nets learn?'
  Rank 1 [doc_4] score=1.2259: Neural networks learn by adjusting weights through backpropagation.
  Rank 2 [doc_3] score=0.0000: Gradient descent is an optimization technique used to minimize the los
  Rank 3 [doc_2] score=0.0000: The BM25 algorithm ranks documents based on term frequency and inverse

Query: 'BM25 term frequency'
  Rank 1 [doc_2] score=2.9625: The BM25 algorithm ranks documents based on term frequency and inverse
  Rank 2 [doc_4] score=0.0000: Neural networks learn by adjusting weights through backpropagation.
  Rank 3 [doc_3] score=0.0000: Gradient descent is an optimization technique used to minimize the los
```

Part 1c: Dense Retrieval — SBERT and ColBERT

```
Loading weights: 100%|▰▰▰
```

```
COMPARISON: BM25 vs SBERT vs Hybrid (for query: 'How do neural nets learn?')
==========================================================================================
Rank   BM25 top doc                                          SBERT top doc
------------------------------------------------------------------------------------------
  #1   Neural networks learn by adjusting weights th         Neural networks learn by adjusting weights th
  #2   Gradient descent is an optimization technique         Gradient descent is an optimization technique
  #3   The BM25 algorithm ranks documents based on t         BERT is a bidirectional encoder trained using
  #4   BERT is a bidirectional encoder trained using         Transformers use self-attention mechanisms to
  #5   Transformers use self-attention mechanisms to         The BM25 algorithm ranks documents based on t

Hybrid Final Ranking:
  #1  Neural networks learn by adjusting weights through backpropagation.
  #2  Gradient descent is an optimization technique used to minimize the loss function.
  #3  BERT is a bidirectional encoder trained using masked language modelling.
  #4  The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  #5  Transformers use self-attention mechanisms to process sequences in parallel.
```

```
Query: 'BM25 term frequency ranking'

Doc      BM25 rank    SBERT rank    RRF Score    Document
------------------------------------------------------------------------------------------
  doc_0  5            5             0.030769     Transformers use self-attention mechanisms to process sequen
  doc_1  4            3             0.031498     BERT is a bidirectional encoder trained using masked languag
  doc_2  1            1             0.032787     The BM25 algorithm ranks documents based on term frequency a
  doc_3  3            4             0.031498     Gradient descent is an optimization technique used to minimi
  doc_4  2            2             0.032258     Neural networks learn by adjusting weights through backpropa

Hybrid Final Ranking:
  #1  doc_2: The BM25 algorithm ranks documents based on term frequency and inverse document frequency.
  #2  doc_4: Neural networks learn by adjusting weights through backpropagation.
  #3  doc_1: BERT is a bidirectional encoder trained using masked language modelling.
  #4  doc_3: Gradient descent is an optimization technique used to minimize the loss function.
  #5  doc_0: Transformers use self-attention mechanisms to process sequences in parallel.
```

Part 1e: Building a Full Hybrid Retriever (Reusable Function)

```
Loading weights: 100%|▰▰▰
```

Part 2a: Re-Ranking with Cross-Encoders

```
modules.json: 100%|| 349/349 [00:00<00:00, 19.4kB/s]
config_sentence_transformers.json: 100%|| 116/116 [00:00<00:00, 10.7kB/s]
README.md: |  10.5k/? [00:00<00:00, 803kB/s]
sentence_bert_config.json: 100%|| 53.0/53.0 [00:00<00:00, 4.77kB/s]
config.json: 100%|| 612/612 [00:00<00:00, 52.2kB/s]
model.safetensors: 100%|| 90.9M/90.9M [00:01<00:00, 206MB/s]
Loading weights: 100%|| 103/103 [00:00<00:00, 176.25it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%|| 350/350 [00:00<00:00, 5.86kB/s]
vocab.txt: |  232k/? [00:00<00:00, 3.33MB/s]
tokenizer.json: |  466k/? [00:00<00:00, 4.59MB/s]
special_tokens_map.json: 100%|| 112/112 [00:00<00:00, 1.80kB/s]
```

```
STAGE 1 — Bi-Encoder Top-5 Candidates
----------------------------------------------------------------------
  #1  [score=0.5273] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2  [score=0.4970] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3  [score=0.3708] BERT is a bidirectional encoder trained using masked language modelling.
  #4  [score=0.3478] Transformers use self-attention mechanisms to process sequences in parallel.
  #5  [score=0.3418] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.
```

```
STAGE 2 — Cross-Encoder Re-Ranking
----------------------------------------------------------------------
  #1  [CE score=3.3522] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2  [CE score=-4.4683] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3  [CE score=-5.4407] Transformers use self-attention mechanisms to process sequences in parallel.
  #4  [CE score=-6.9237] BERT is a bidirectional encoder trained using masked language modelling.
  #5  [CE score=-10.4939] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.

--- Before vs After Re-Ranking ---
Bi-Encoder Rank 1:            The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
Cross-Encoder Rank 1:         The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
```

Part 2b: Query Expansion

```
Original Query:
  'How does attention work in language models?'

Hypothetical Document (generated by Gemini):
  In language models, attention is a mechanism that allows the model to focus on specific parts of the input sequence when generating output. This is achieved through the use of atter
```

```
Query: 'How does attention work in language models?'

Raw Query Retrieval (Top 3):
  #1 [score=0.5273] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2 [score=0.4970] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3 [score=0.3708] BERT is a bidirectional encoder trained using masked language modelling.

HyDE Retrieval (Top 3):
  #1 [score=0.6191] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  #2 [score=0.4957] Large language models are trained on massive text corpora to learn general-purpose representations.
  #3 [score=0.4092] Fine-tuning adapts a pre-trained model to a specific downstream task using task-specific data.
```

```
Generated query variants:
  (original): How does attention work in language models?
  (variant 1): What is the mechanism behind attention in language models that enables them to focus on specific parts of the input?
  (variant 2): How does the attention mechanism in language models allow them to selectively weigh and combine different parts of the input to generate the output?
  (variant 3): Can you explain the role of attention in language models and how it helps them to better understand and respond to user queries?

Multi-Query retrieved 5 unique documents (union of all variants):
  [doc_8] The attention mechanism computes a weighted sum of value vectors based on query-key similarity.
  [doc_9] Large language models are trained on massive text corpora to learn general-purpose representations.
  [doc_1] BERT is a bidirectional encoder trained using masked language modelling.
  [doc_0] Transformers use self-attention mechanisms to process sequences in parallel.
  [doc_5] Retrieval Augmented Generation combines a retriever with a language model to produce grounded answers.
```

Part 2c: Full Production Pipeline — Query Expansion → Hybrid → Re-Rank → Generate

```
Loading weights: 100%|▰▰▰
```

```
Original weight W:      64x64 = 4,096 parameters
LoRA matrices A+B:      4x64 + 64x4 = 512 parameters
LoRA overhead:          12.50% of original

Original output:  [ 0.0417  -0.05841  0.07145  0.00424]
LoRA output:      [ 0.0417  -0.05841  0.07145  0.00424]
(outputs identical at init because B=0, so ΔW = B@A = 0)
```

```
Loading a tiny model for PEFT demo (distilgpt2)...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab a
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
config.json: 100% ||||||
```

```
Linear Quantization Demo (FP32 → INT8 → FP32)
==================================================================
Scale factor S = 0.007111
Zero point   Z = -95

Index    FP32         INT8       Reconstructed    Error
------------------------------------------------------------------
  [0]     0.496714     -25         0.497787   0.001073
  [1]    -0.138264    -114        -0.135114   0.003151
  [2]     0.647689      -4         0.647123   0.000566
  [3]     1.523030     119         1.521805   0.001225
  [4]    -0.234153    -128        -0.234671   0.000518
  [5]    -0.234137    -128        -0.234671   0.000534
  [6]     1.579213     127         1.578695   0.000518
  [7]     0.767435      13         0.768014   0.000579

Max quantization error: 0.003151
Mean quantization error: 0.001020

Memory: FP32=32 bytes → INT8=8 bytes (4.0x reduction)
```

```
Model Size Comparison for Common LLMs
==================================================================
Model                Params       FP32 (GB)    FP16 (GB)    INT8 (GB)    INT4 (GB)
------------------------------------------------------------------
distilgpt2           0.07B        0.25         0.12         0.06         0.03
GPT-2                0.12B        0.44         0.22         0.11         0.05
BERT-Base            0.11B        0.41         0.20         0.10         0.05
Llama-3.2-1B         1.00B        3.73         1.86         0.93         0.47
Llama-3.1-8B         8.00B        29.80        14.90        7.45         3.73
Llama-3.1-70B        70.00B        260.77       130.39       65.19        32.60
```

```
Quantization Trade-off (100 weights)
Bits     MSE (error)        Size (bytes)    Compression
-----------------------------------------------------
  32       0.23258863          400.0            1.0x
  16       0.00000000          200.0            2.0x
  8        0.00003263          100.0            4.0x
  4        0.00870766          50.0             8.0x
  2        0.19430648          25.0             16.0x
/tmp/ipykernel_11604/792760889.py:16: RuntimeWarning: invalid value encountered in cast
  q_vals = np.clip(np.round(W / S + Z), q_min, q_max).astype(np.int32)
```

Part 3d: Mixture of Experts (MoE)

```
Defined 5 experts: ['math', 'code', 'creative', 'science', 'general']
```

```
Router Test:
------------------------------------------------------------
  🔢 [math      ] What is the derivative of x² + 3x?
  💻 [code      ] Write a Python function to reverse a linked list.
  ✍️ [creative  ] Write a short poem about autumn leaves.
  🔬 [science   ] How does photosynthesis work?
  🤖 [general   ] What time is it in Tokyo?
```

```
🔢 Expert: MATH
Query:  What is the integral of sin(x)dx?
Answer: To find the integral of $\sin(x)$, we can use the following formula:

$$\int \sin(x) \, dx = -\cos(x) + C$$

where $C$ is the constant of integration.

To derive this formula, we can use the following steps:

1. Recall the definition of the derivative of the cosine function:

$$\frac{d}{dx} \cos(x) = -\sin(x)$$

2. Integrate both sides of the equation with respect to $x$:

$$\int \frac{d}{dx} \cos...
======================================================================
💻 Expert: CODE
Query:  Write a recursive function to compute Fibonacci numbers in Python.
Answer: **Fibonacci Recursive Function in Python**
```python
def fibonacci(n: int) -> int:
    """
    Compute the nth Fibonacci number recursively.

    Args:
        n (int): The position of the Fibonacci number to compute.

    Returns:
        int: The nth Fibonacci number.

    Raises:
        ValueError: If n is a negative integer.
    """
    if n < 0:
        raise ValueError("n must be a non-nega...
======================================================================
```
```

```
💻 Expert: CODE
Query:  How do transformer models use attention mechanisms?
Answer: **Transformer Model with Attention Mechanism**

The Transformer model, introduced in the paper "Attention Is All You Need" by Vaswani et al. in 2017, revolutionized the field of natural language processing (NLP) by using self-atter

```python
impo...
====================================================================
```

```
Query: 'Explain recursion.'
====================================================================

💻 CODE Expert:
**Recursion in Programming**
================================

Recursion is a fundamental concept in programming where a function calls itself repeatedly until it reaches a base case that stops the recursion. This technique is used to solve probl
--------------------------------------------------------------------

✍️ CREATIVE Expert:
In the whispering twilight of a winter's eve, a lone mathematician sat huddled by the fireplace, lost in the labyrinthine world of numbers. Her eyes, bright as the stars on a clear ni

Recursion, a word that echoed thro...
--------------------------------------------------------------------

🔬 SCIENCE Expert:
Recursion is a fundamental concept in computer science and mathematics that can be a bit tricky to grasp at first, but don't worry, I'm here to help you understand it with some analog

**What is Recursion?**

Recursion is a programming technique where a function calls its...
--------------------------------------------------------------------
```
```